

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

04/09/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with support for the *compare-and-swap* primitive.

This primitive will be available through a new statement called **cswap** having the following syntax:

```
cswap (<var. 1>, <exp. 1>, <exp. 2>, <var. 2>);
```

The statement, when executed, performs the following operations. First, it checks whether variable $\langle var. 1 \rangle$ is equal to expression $\langle exp. 1 \rangle$. If the two are indeed equal, the statement assigns the value of expression $\langle exp. 2 \rangle$ to $\langle var. 1 \rangle$, and assigns the *old* value of $\langle var. 1 \rangle$ to variable $\langle var. 2 \rangle$. On the contrary, if the value of $\langle var. 1 \rangle$ is different from the value of $\langle exp. 1 \rangle$, $\langle var. 1 \rangle$ is left unmodified, and its value is also assigned to $\langle var. 2 \rangle$. If both $\langle var. 1 \rangle$ and $\langle var. 2 \rangle$ refer to the same variable, the behavior of the statement is undefined (in other words the implementation doesn't need to handle this case).

The following short program exemplifies the operation of the `cswap` statement. Initially the program declares two variables *a* and *b*, with *a* immediately initialized to 10 and *b* left uninitialized. Then, the `cswap` statement is used to compare *a* with 10. Since *a* is indeed equal to 10, *a* is assigned the value 11, and *b* is assigned the old value of *a*, therefore *b* = 10. At this point `cswap` is used again to compare *a* with 12. However, *a* is equal to 11 and not 12, therefore *a* is left unmodified and *b* is assigned the value 11.

```
int a, b;
a = 10;

cswap(a, 10, 11, b); // compare successful
// a == 11, b == 10

cswap(a, 12, 15*2, b); // compare failed
// a == 11, b == 11
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (3 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (18 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..de2169c 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"cswap"      { return CSWAP; }

{ID}          { yyval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yyval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..946de94 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+%token CSWAP

%token <label> DO
%token <while_stmt> WHILE
@@ -248,6 +249,7 @@ statement : assign_statement SEMI { /* does nothing */ }
           | control_statement      { /* does nothing */ }
           | read_write_statement SEMI { /* does nothing */ }
           | SEMI { gen_nop_instruction(program); }
+           | cswap_statement SEMI
+
+
;

control_statement : if_statement { /* does nothing */ }
@@ -260,6 +262,36 @@ read_write_statement : read_statement { /* does nothing */ }
                     | write_statement { /* does nothing */ }
;

+cswap_statement: CSWAP LPAR IDENTIFIER COMMA exp COMMA exp COMMA IDENTIFIER RPAR
+ {
+     // Get the registers associated to both variables.
+     int r_var1 = get_symbol_location(program, $3, 0);
+     int r_var2 = get_symbol_location(program, $9, 0);
+     // Generate code to assign the value of the first variable to the second
+     // variable.
+     gen_addi_instruction(program, r_var2, r_var1, 0);
+     // Generate code to compare the first variable with the first expression,
+     // and if the two are different, jump ahead skipping over the assignment
+     // to the first variable.
+     if ($5.expression_type == REGISTER) {
+         gen_sub_instruction(program, REG_0, r_var1, $5.value, CG_DIRECT_ALL);
+     } else {
+         gen_subi_instruction(program, REG_0, r_var1, $5.value);
+     }
+     t_axe_label *l_different = newLabel(program);
+     gen_bne_instruction(program, l_different, 0);
+     // Generate code to assign the value of the second expression to the first
```

```

+      // variable.
+      if ($7.expression_type == REGISTER) {
+          gen_addi_instruction(program, r_var1, $7.value, 0);
+      } else {
+          gen_addi_instruction(program, r_var1, REG_0, $7.value);
+      }
+      // Generate the label used to skip over the assignment just generated.
+      assignLabel(program, l_different);
+  }
+;
+
+  assign_statement : IDENTIFIER LSQUARE exp RSQUARE ASSIGN exp
+  {
+      /* Notify to 'program' that the value $6
diff --git a/tests/cswap/cswap.src b/tests/cswap/cswap.src
new file mode 100644
index 0000000..214321a
--- /dev/null
+++ b/tests/cswap/cswap.src
@@ -0,0 +1,11 @@
+int a, b, c;
+a = 10;
+c = 11;
+
+cswap(a, 10, c, b); // compare successful
+write(a); // 11
+write(b); // 10
+
+cswap(a, c+1, 15*2, b); // compare failed
+write(a); // 11
+write(b); // 11

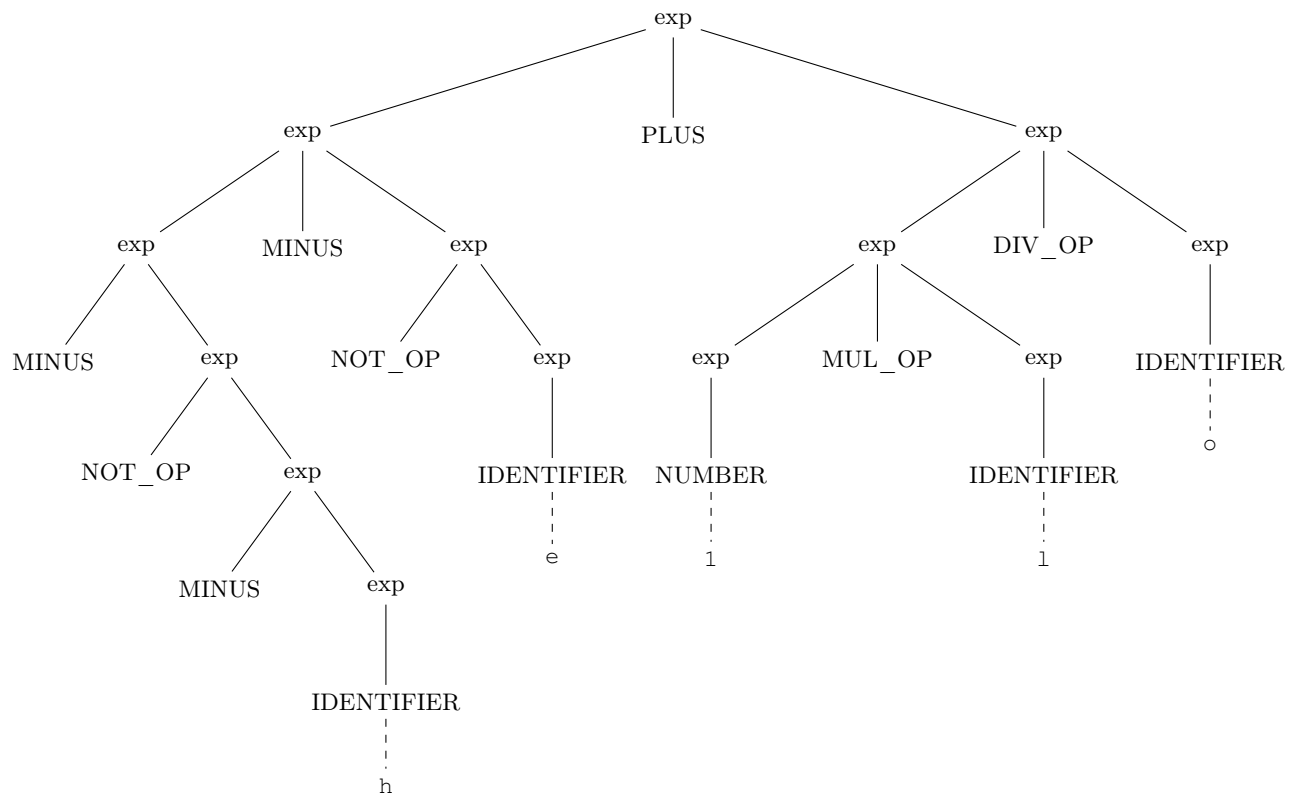
```

4. Given the following Lance code snippet:

- ! - h - ! e + 1 * 1 / o

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the exp nonterminal*. (5 points)

Solution



5. **(Bonus)** Briefly explain in plain English words how to modify the solution given to the questions about the `cswap` statement in order to transform it into an expression operator. The fourth argument is removed, and the operator itself now returns the previous value of the first variable, like in the following example.

```
int a, b;  
a = 10;  
  
b = cswap(a, 10, 11);  
// a == 11, b == 10  
  
b = 3 - cswap(a, 12, 15*2) * 4;  
// a == 11, b == 3-11*4 == -41
```